

University of Exeter

Department of Computer Science

ECM3412

Programming Task and Report

Candidate number: [740007415]

Abstract

This report evaluates Ant Colony Optimisation (ACO) for a fixed-bin load-balancing problem. Each ant assigns every item to one bin and fitness is the difference between the heaviest and lightest bin. Two prescribed instances were tested with population sizes $p \in \{10, 50\}$ and evaporation rates $e \in \{0.05, 0.30\}$. Each setting used five independent trials and 10,000 fitness evaluations. In the required grid, BPP1 favoured $p = 10, e = 0.30$ (mean final fitness 1091.60), whereas BPP2 favoured $p = 50, e = 0.30$ (mean 143855.55). A targeted evaporation sweep retained the BPP1 result but improved BPP2 to mean 99932.25 at $p = 50, e = 0.60$. Population size and evaporation interact through sampling breadth, update frequency, and pheromone memory.

1 Introduction

The classical bin-packing problem assigns weighted items to capacity-constrained bins while minimising the number of bins. This project studies a related fixed-bin load-balancing problem, also closely related to multiway number partitioning. Every item must be assigned to one of a prescribed number of bins, and the objective is

$$d = \max_j L_j - \min_j L_j,$$

where L_j is the load of bin j. Lower d is better, with zero representing perfect balance.

The project implements ACO using a construction graph whose rows represent items and columns represent bins. Ants select bins according to pheromone values; each route deposits $100/(d + 1)$ on its selected edges, all pheromone is multiplied by $(1 - e)$, and a trial stops after 10,000 fitness evaluations. BPP1 contains 600 items and 12 bins with $w(i) = 2i + 1$; BPP2 contains 400 items and 40 bins with $w(i) = i^2/4$. The experiments investigate how population size and evaporation affect performance.

2 Literature Review

Bin packing is a family of combinatorial optimisation problems with objectives including bin count, unused capacity, and load balance. Classical one-dimensional bin packing is NP-hard, making heuristic and metaheuristic methods useful for large instances [9, 5]. A recent survey of one-dimensional bin-packing research also shows that population-based and hybrid metaheuristics remain active areas of study [8]. However, most published studies minimise the number of capacity-constrained bins, whereas this project fixes the number of bins and minimises load range. This difference is important when comparing algorithms.

2.1 Ant Colony Optimisation

ACO is a constructive metaheuristic inspired by pheromone-mediated ant foraging. Artificial ants probabilistically construct solutions, good routes deposit pheromone, and evaporation prevents all future search from following old decisions [4, 3]. Positive feedback supports exploitation, while probabilistic construction provides exploration.

In this project, pheromone entry τ_{ij} represents assigning item i to bin j. An ant makes one decision for each item, so a complete route is a complete assignment. The implementation uses no local heuristic: the probability of choosing a bin is proportional only to the corresponding pheromone row. After calculating d, every used edge receives $100/(d+1)$ and the whole graph is evaporated. This directly follows the brief and makes the effects of p and e experimentally visible.

Levine and Ducatelle apply ACO to bin-packing and cutting-stock problems. They report that the "pure ACO approach can compete with existing evolutionary methods" and that adding local search can improve performance for some problem classes [6]. Their work supports ACO's suitability for discrete packing, but also shows that representation and local improvement matter. More sophisticated variants such as MAX–MIN Ant System restrict pheromone values and update rules to reduce stagnation [10]; those additions are outside this coursework implementation.

2.2 Genetic Algorithm

Genetic Algorithms use selection, crossover, and mutation to evolve a population. A bin-packing chromosome may encode direct item-to-bin assignments, item permutations, or groups of items. Generic crossover can destroy useful groups or create poor assignments, so effective methods require problem-specific operators or repair.

Yakawa, Iima and Sannomiya design crossover to inherit item combinations within bins and mutation to encourage capacity-feasible packing. They aim to generate "diverse offsprings" and report frequent optimal solutions on their benchmarks [11]. Chan *et al.* likewise show that a GA can incorporate quality-related objectives rather than treating bin count as the only measure of solution quality [2]. A GA could represent this project's solution directly, but its crossover would need to preserve balanced groups rather than only feasibility. Compared with ACO, a GA explicitly retains competing complete solutions; its disadvantages are operator design and possible repair overhead. A fair comparison would also require the same fitness-evaluation budget.

2.3 Particle Swarm Optimisation

PSO models social information sharing between particles. Standard PSO is continuous, whereas a bin assignment is a discrete categorical decision. A packing PSO therefore needs a discrete representation, a rule for assignment changes, and often repair or local search.

Liu *et al.* propose an evolutionary PSO for a two-dimensional multiobjective problem involving bin usage and load balance. Their Pareto-based method is relevant because it

treats balance explicitly rather than only minimising bin count [7]. PSO may spread good assignments rapidly through a population, but velocity is not naturally defined for item-to-bin choices. Implementing it would therefore require more problem-specific design than this ACO.

2.4 Other Nature-Inspired Approaches

Simulated Annealing (SA) is a physics-inspired method based on controlled cooling. It accepts some worsening moves early in the search and becomes more selective as temperature decreases. Rao and Iyengar report significant improvement over the heuristics they tested on a bin-packing variant [9]. For this project, an SA neighbourhood could move or swap items between bins and directly evaluate the change in d. Its results would depend heavily on the cooling schedule and neighbourhood.

Hybrid methods combine a population or stochastic search with domain-specific improvement. Alvim *et al.* combine lower bounds, load redistribution, and tabu search for one-dimensional bin packing [1]. Hemmelmayr, Hirsch and Schmid combine variable neighbourhood search with lower bounds and dynamic programming for variable-sized bin packing [5]. These are not pure nature-inspired algorithms, but they are important comparators because they show that a well-designed neighbourhood can be as important as the high-level metaphor. The recent survey by Munien and Ezugwu similarly identifies hybridisation as an important direction for improving bin-packing metaheuristics [8].

2.5 Critical Analysis

The published algorithms are not directly comparable because they often use different objectives, constraints, dimensions, and benchmarks. A method that minimises the number of capacity-constrained bins may not be best for a fixed number of bins and a load-range objective. Table 1 compares the main approaches specifically against the problem representation used in this project.

ACO is the most direct fit for the coursework because every item-to-bin assignment is a clear construction decision and pheromone can be attached directly to that decision. It is also transparent: the probability rule, reward, evaporation rate, and evaluation budget can be stated and tested explicitly. The required implementation is intentionally basic; it has no local search, pheromone bounds, heuristic information, or elitist global-best retention. These omissions make it less sophisticated than several published hybrids but provide a controlled experiment on population size and evaporation.

The literature therefore exposes a specific gap addressed by this study. Published work demonstrates that ACO, evolutionary algorithms, PSO, and hybrid methods can perform well on several bin-packing variants, but those results generally concern minimum-bin or multidimensional objectives. This project instead holds the number of bins fixed and evaluates load-range minimisation under an identical 10,000-evaluation budget. Classical bin-packing results therefore motivate algorithm design but do not establish performance on this objective. A direct comparison would require the same problem instances, fitness definition, and evaluation budget.

Table 1: Critical comparison of approaches for the fixed-bin balancing problem.

Approach	Representation	Strength for this project	Limitation
ACO	Item–bin pheromone graph	Directly represents each discrete assignment and learns from complete routes.	Positive feedback can cause stagnation; performance depends on evaporation and update policy.
GA	Assignment, permutation, or grouping chromosome	Crossover can preserve useful item groups and maintain several complete solutions.	Generic crossover may destroy balance; repair and grouping operators are difficult to design.
PSO	Discrete assignment particles	Good solutions can spread rapidly through social information sharing.	Continuous velocity has no natural meaning for categorical bin choices.
SA / hybrid search	Item moves, swaps, and neighbourhoods	Local moves directly target the load-range objective.	Cooling, neighbourhood design, and local-search cost strongly affect performance.

The study's contribution is not a new metaheuristic, but a controlled empirical analysis of population size and evaporation in a basic ACO on two structured weight sequences. A hybrid ACO, grouping GA, or discrete PSO might perform better, but that claim would require a new implementation and equal evaluation budget. A simple greedy load-balancing heuristic would also provide a useful non-nature-inspired baseline for judging whether the extra search machinery improves solution quality.

3 Description of Results

3.1 Experimental Setup

The construction graph contains one row per item and one column per bin. Pheromone values are initialised randomly in $[0, 1)$. Roulette-wheel selection chooses each bin with probability proportional to its pheromone value. For each population, all routes are evaluated, their deposits are summed, and evaporation follows. The required settings were $(p, e) = (50, 0.05), (50, 0.30), (10, 0.05), (10, 0.30)$. Each setting was run five times on BPP1 and five times on BPP2, producing 40 trials. Every trial used a distinct seed and exactly 10,000 evaluations. Results below use the best ant in the final population, as required by the brief; best-seen fitness is retained only as a diagnostic.

3.2 Experimental Results

BPP1 strongly favours $p = 10, e = 0.30$, while BPP2 favours $p = 50, e = 0.30$. The common result is that $e = 0.30$ outperforms $e = 0.05$ for both population sizes and both problems.

Table 2: Required trials. Mean, standard deviation, best and worst final fitness; lower is better.

Problem	p	e	Mean	SD	Best	Worst
BPP1	50	0.05	8163.00	654.19	7550.00	9138.00
BPP1	50	0.30	6344.00	768.19	5227.00	7073.00
BPP1	10	0.05	9141.40	1763.18	7005.00	11319.00
BPP1	10	0.30	1091.60	253.82	778.00	1321.00
BPP2	50	0.05	172004.05	2960.86	167253.25	174446.50
BPP2	50	0.30	143855.55	12667.98	126874.75	158713.25
BPP2	10	0.05	177969.70	26034.03	143774.50	199838.50
BPP2	10	0.30	146515.70	11286.18	129393.75	157343.25

Fitness scales differ substantially between the problems, so values are compared only within each problem.

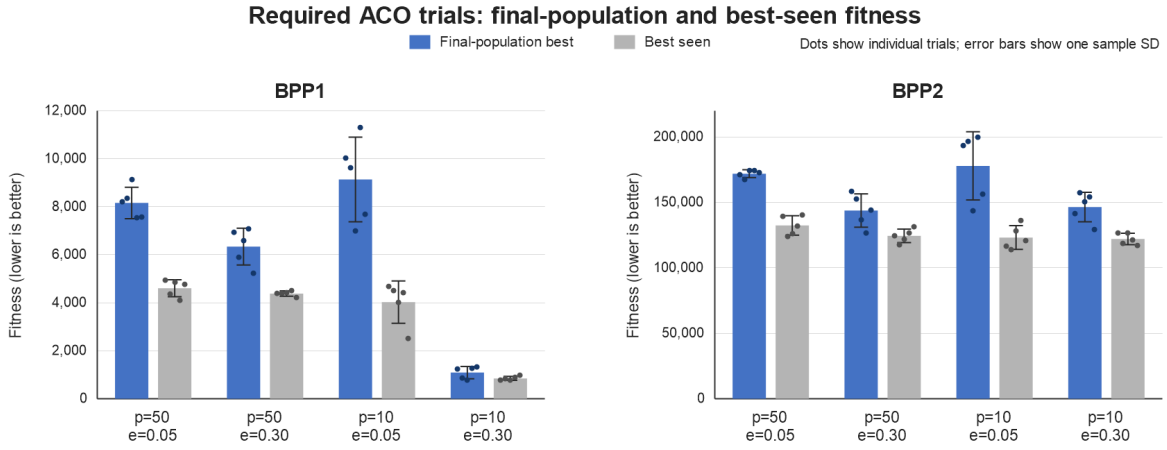


Figure 1: Mean final-population and best-seen fitness across the required trials. Dots show individual trials and error bars show one sample standard deviation.

4 Discussion

4.1 Answer to Question 1

Among the required settings, the best combination is problem-dependent. BPP1 is best with $p = 10, e = 0.30$, giving mean final fitness 1091.60. BPP2 is best with $p = 50, e = 0.30$, giving mean final fitness 143855.55. The shared feature is high evaporation within the

required grid, while the preferred population size depends on the instance. Section 5 tests whether $e = 0.30$ remains best when the range is extended.

4.2 Answer to Question 2

At $e = 0.05$, 95% of pheromone is retained after each update. Since the initial table is random, early decisions can retain an accidental bias for too long. At $e = 0.30$, only 70% is retained, allowing later populations to replace misleading decisions. The observed advantage of $e = 0.30$ is therefore consistent with a shorter pheromone memory, although the experiment does not directly measure exploration.

The size of the effect supports this interpretation. Increasing e from 0.05 to 0.30 reduces mean final fitness by 22.28% for $p = 50$ on BPP1 and by 88.06% for $p = 10$. On BPP2, the corresponding improvements are 16.37% and 17.67%. The especially large BPP1 improvement for $p = 10$ suggests that frequent updates and rapid pheromone replacement work particularly well for that instance.

Population size changes more than the number of assignments sampled together. Under 10,000 evaluations, $p = 10$ gives 1,000 pheromone updates, while $p = 50$ gives 200. The contribution of an existing pheromone value after t further evaluations is proportional to $(1 - e)^{t/p}$. Its approximate e-folding memory is consequently $-p/\ln(1 - e)$ evaluations: 195 and 28 evaluations for $p = 10$ at $e = 0.05$ and 0.30, compared with 975 and 140 for $p = 50$. Moreover, deposits from all ants are summed before each evaporation step, so a 50-ant update aggregates five times as many routes as a 10-ant update. Population size and evaporation therefore interact through sampling breadth, update frequency, reinforcement magnitude, and memory. BPP1 appears to benefit from the shortest-memory setting, whereas BPP2 benefits slightly from the broader 50-ant sample. These explanations remain tentative because each setting has only five trials.

4.3 Answer to Question 3

A larger population samples more complete assignments before each update, so its reinforcement reflects more routes, but it performs fewer update and evaporation cycles under a fixed evaluation budget. A smaller population updates more frequently from fewer routes. The unusually strong BPP1 result at $p = 10, e = 0.30$ is consistent with rapid replacement of earlier pheromone, while the smaller difference between the two $e = 0.30$ populations on BPP2 suggests that broader sampling was more useful there.

Low evaporation gives longer memory but can preserve poor early preferences. High evaporation shortens memory and permits quicker changes in selection probabilities; $e = 0.30$ improved both problems in the required grid. This result should not be generalised beyond the tested range because still higher evaporation may discard useful information too quickly. The figure also shows that mean best-seen fitness is lower than mean final-population fitness for most settings. The required implementation reports the best ant in the final population and does not preserve an elite global solution, so finding a good route earlier does not guarantee that it remains at termination.

4.4 Answer to Question 4

A hybrid ACO with local search is the strongest candidate for improvement because a local move or swap can directly reduce the load range after an ant constructs a route. Levine and Ducatelle found that local search strengthened ACO on some packing classes [6]. A grouping GA could also preserve useful item groups, while PSO would require a discrete assignment model and repair mechanism. SA or tabu search could provide effective local improvement.

However, the reviewed studies use different problem definitions and budgets. It is therefore not possible to claim that one alternative would definitely outperform this implementation without implementing and testing it under the same 10,000-evaluation limit.

5 Further Experiments

The additional work had two stages. First, the intermediate setting $p = 25, e = 0.15$ was tested on both problems. Second, evaporation was varied while holding population size at the best required value for each problem: $p = 10$ for BPP1 and $p = 50$ for BPP2. The sweep tested $e \in \{0.15, 0.45, 0.60\}$ and compared it with the required $e = 0.30$ result. Every additional setting used five new seeds and the same 10,000-evaluation limit.

Table 3: Additional experiments with required $e = 0.30$ baselines included for comparison.

Problem	p	e	Mean	SD	Best	Worst
BPP1	25	0.15	8105.20	1586.77	5685.00	9595.00
BPP1	10	0.15	2599.80	353.18	2208.00	3050.00
BPP1	10	0.30*	1091.60	253.82	778.00	1321.00
BPP1	10	0.45	1868.40	514.65	1075.00	2243.00
BPP1	10	0.60	3440.80	1301.78	2188.00	4837.00
BPP2	25	0.15	161330.45	6482.13	153290.50	167263.75
BPP2	50	0.15	153351.65	16703.94	129660.75	171427.00
BPP2	50	0.30*	143855.55	12667.98	126874.75	158713.25
BPP2	50	0.45	133071.85	9281.93	119660.50	142475.75
BPP2	50	0.60	99932.25	10403.21	90728.50	114202.75

* Required setting, repeated for comparison.

The midpoint setting did not beat the best required result on either problem. The targeted sweep then showed that the effect of higher evaporation differs by instance. On BPP1, $e = 0.30$ remained best; raising evaporation to 0.45 or 0.60 worsened the mean, consistent with useful pheromone being removed too quickly. On BPP2, performance improved as evaporation increased across the tested values. The $e = 0.60$ mean was 99932.25, a 30.53% reduction from the required $e = 0.30$ mean. Its worst trial (114202.75) was also better than the best $e = 0.30$ trial (126874.75).

These results show that the preferred memory length depends on the problem and that $e = 0.30$ is not a universal optimum. The sweep uses only five trials per setting and one population size per problem, so it does not resolve other interactions. A larger factorial

experiment with more repetitions and confidence intervals would be needed for statistical claims.

6 Conclusions

The required ACO experiments show that parameter choice and problem structure interact. BPP1 performed best with $p = 10, e = 0.30$, while BPP2 performed best with $p = 50, e = 0.30$. The population size also changes update frequency and the effective pheromone memory, so its effect cannot be separated from the update schedule.

The additional experiments refined rather than merely repeated this conclusion. The midpoint setting did not improve either problem, and BPP1 deteriorated above $e = 0.30$. In contrast, BPP2 improved substantially at $p = 50, e = 0.60$. Higher evaporation can therefore help when shorter memory is useful, but the preferred rate depends on the instance and should be established experimentally.

The literature suggests that local search, grouping-aware genetic operators, or discrete swarm operators could improve the algorithm, but such comparisons would need separate implementations and equal evaluation budgets. Overall, the basic ACO provides a transparent method for studying fixed-bin load balancing, and the results demonstrate that no single population size should be assumed to be optimal across different problem structures.

References

[1] Alvim, A. C. F., Ribeiro, C. C., Glover, F., and Aloise, D. J. A hybrid improvement heuristic for the one-dimensional bin packing problem. *Journal of Heuristics 10*, 2 (2004), 205–229.

[2] Chan, T. S., Au, K. C., Chan, L. Y., and Lau, T. L. Using genetic algorithms to solve quality-related bin packing problem. *Robotics and Computer-Integrated Manufacturing 23*, 1 (2007), 71–81.

[3] Dorigo, M., Di Caro, G., and Gambardella, L. M. Ant algorithms for discrete optimization. *Artificial Life 5*, 2 (1999), 137–172.

[4] Dorigo, M., Maniezzo, V., and Colorni, A. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 26*, 1 (1996), 29–41.

[5] Hemmelmayr, V., Hirsch, R. F., and Schmid, V. Variable neighbourhood search for the variable sized bin packing problem. *Computers and Operations Research 39*, 5 (2012), 1097–1108.

[6] Levine, J., and Ducatelle, F. Ant colony optimisation and local search for bin-packing and cutting stock problems. *Journal of the Operational Research Society 55*, 7 (2004), 705–716.

[7] Liu, D. S., Tan, K. C., Huang, S. Y., Goh, C. K., and Ho, W. K. On solving multiobjective bin packing problems using evolutionary particle swarm optimization. *European Journal of Operational Research 190*, 2 (2008), 357–382.

[8] Munien, C., and Ezugwu, A. E. Metaheuristic algorithms for one-dimensional bin-packing problems: A survey of recent advances and applications. *Journal of Intelligent Systems 30*, 1 (2021), 636–663.

[9] Rao, R. L., and Iyengar, S. S. Bin-packing by simulated annealing. *Computers and Mathematics with Applications 27*, 5 (1994), 71–82.

[10] Stutzle, T., and Hoos, H. H. Max–min ant system. *Future Generation Computer Systems 16*, 8 (2000), 889–914.

[11] Yakawa, T., Iima, H., and Sannomiya, N. Proposition of genetic algorithm for bin packing problems. *Transactions of the Society of Instrument and Control Engineers 41*, 3 (2005), 274–282.